`spring.datasource.url` property and specify the same URL, replacing `localhost` with the PostgreSQL container name: `polar-postgres`. Using another environment variable, we can also enable the `testdata` Spring profile to trigger the creation of test data in the catalog:

```
$ docker run -d \
    --name catalog-service \
    --net catalog-network \
    -p 9001:9001 \
    -e SPRING_DATASOURCE_URL=
➥jdbc:postgresql://polar-postgres:5432/polardb_catalog \
    -e SPRING_PROFILES_ACTIVE=testdata \
    catalog-service
```

That's quite a long command, isn't it? You won't use the Docker CLI for long, though, I promise. Later in the chapter I'll introduce Docker Compose.

Open a Terminal window, call the application, and verify that it works correctly, as it did in chapter 5:

```
$ http :9001/books
```

When you're done, remember to delete both containers:

```
$ docker rm -f catalog-service polar-postgres
```

The approach you've just followed is perfectly fine for experimenting with Docker in your development environment and understanding how images work, but you'll need to consider several aspects before achieving a production-grade image. That's the topic of the next section.

6.2.3 Building container images for production